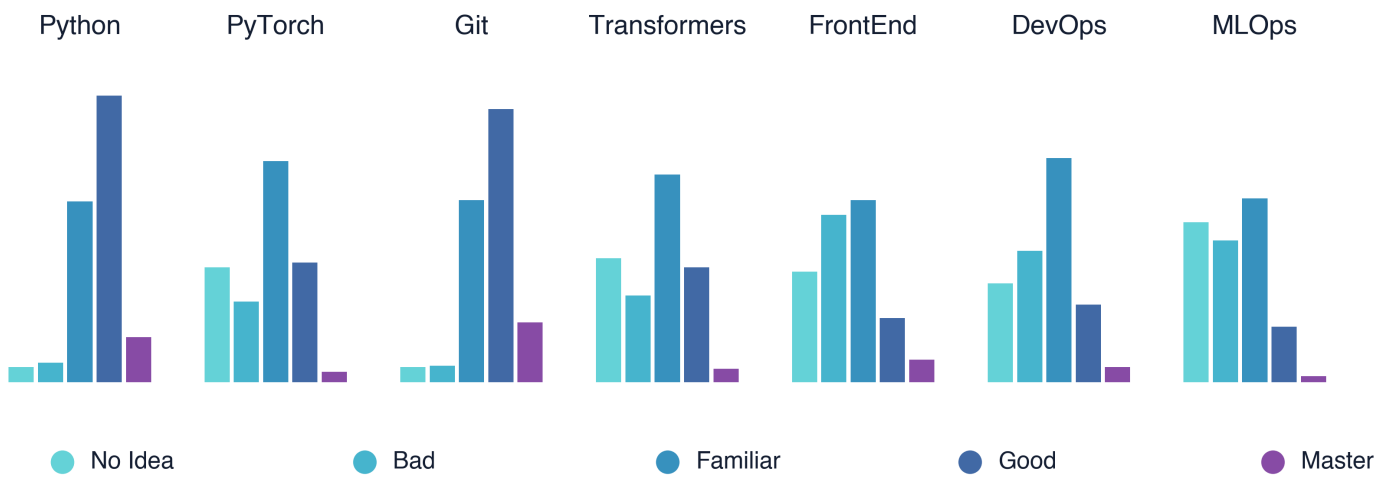
Session 1: Foundations of Transformer Architecture

Session 1 - Foundations of the Transformer Architecture

Welcome to the EAG V3 Course.

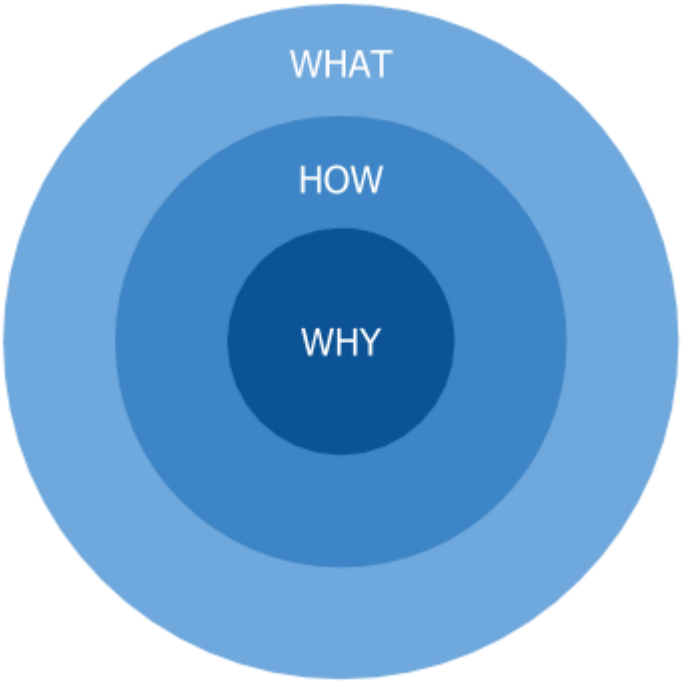
Let's head over to **Syllabus (https://canvas.instructure.com/courses/12633376/pages/syllabus)** first to see what we've got!

We have a lot to cover, and given that our students have this level of mastery on different topics,



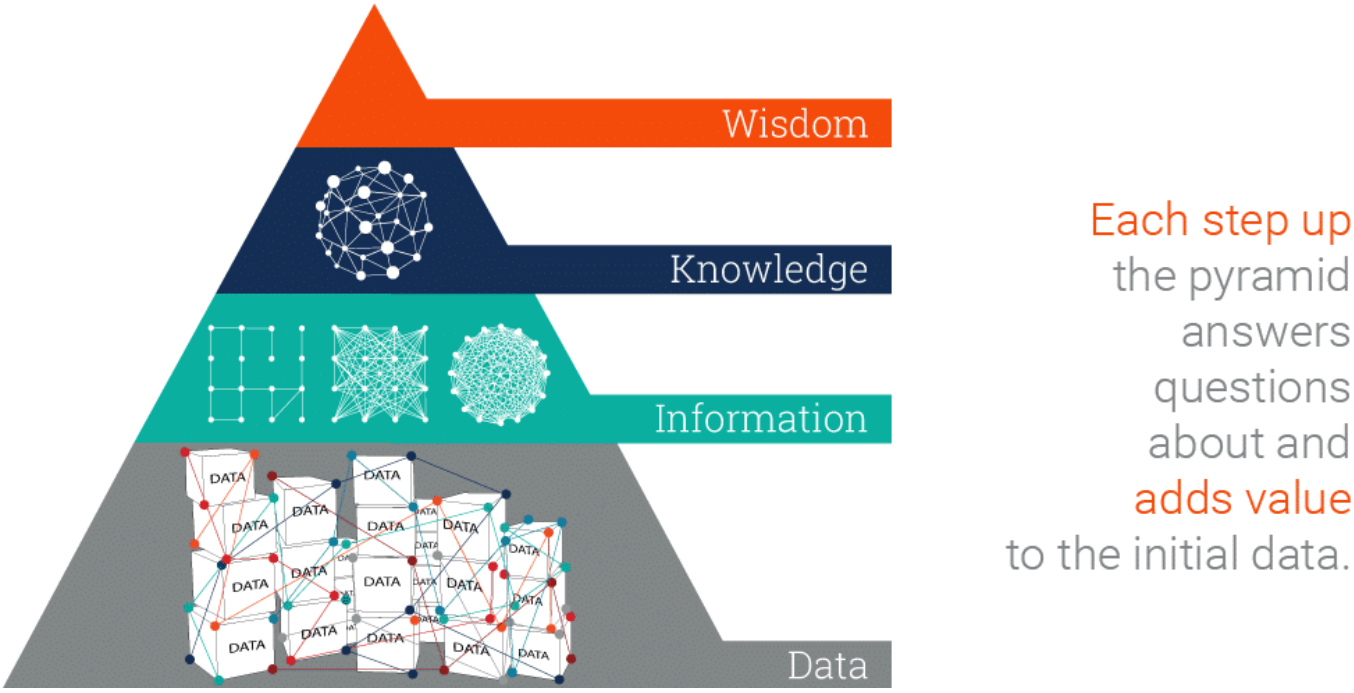
we'll sprinkle these topics throughout our sessions.

But, before we start!



This is **Simon Sinek's Golden Circle (https://www.youtube.com/watch?v=u4ZoJKF_VuA)** . Though I agree with him, we'll follow the reverse of what he says.

And the reason is:



My task in this course is to identify "what's", "how's" and "why's" for you. The speed at which things are growing, identifying this is extremely important.

You're going to spend a LOT of time in understanding the why's so it's only going to be for super special things.

Also, my job here is to simplify things for you and to make you believe that these things aren't difficult. Once you remove the GRE, or Consulting Company Driven or Research/Academia driven english out, things are super obvious, just by their names.

Let me scare you a little

- The programming languages, Data Formats, etc. you're supposed to know:
  - Python

  - JavaScript/TypeScript
  - SQL
  - Bash
  - JSON
  - YAML
  - CSV
  - Markdown
  - HDF5
  - ONNX/GGUF
- Core Concepts that are extremely important to master:
  - LLMs & Transformers (architecture, fine-tuning, LoRA, RAG)
  - Prompt Engineering & Optimization
  - Memory Systems (FAISS, Milvus)
  - Planning Systems (ReACT, decision chains)
  - Tool Use & Function Calling
  - Web APIs (REST & GraphQL)
  - gRPC
  - Browser Automation (Playwright, Puppeteer, Selenium, etc)
  - Docker
  - CI/CD Pipelines
  - Gradio, FastAPI

Good news!

- These things are designed by humans, and can be understood by humans easily
- Things like ClaudeCode, Cursor and ChatGPT would make our lives easier
- We'll learn them as they are required, and most importantly
- You do not need to master ALL of them. No body can!

Let's start with Cursor (or AntiGravity/ClaudeCode/Codex)

Cursor is a code completion tool:

```
vscode > src > vs > workbench > contrib > aiCpp > electron-sandbox > TS cppActions.ts > AcceptCppSuggestionAction > run
216  export class AcceptCppSuggestionAction extends Action2 {
220    constructor() {
228        menu: {
229          id: CppDiffPeekView.TitleMenu,
230          order: 1,
231        },
232      });
233    }
234    ⌘L to chat, ⌘K to generate
235  }
236
237  registerAction2(AcceptCppSuggestionAction);
238
239  function getGhostTextControllerFromAccessor(accessor: ServicesAccessor): GhostTextController | undefined {
240    const codeEditorService = accessor.get(ICodeEditorService);
241    const currentEditor = codeEditorService.getActiveCodeEditor() || codeEditorService.getFocusedCodeEditor();
242    if (!currentEditor) {
243      return;
244    }
245    const controller = GhostTextController.get(currentEditor);
246    return controller || undefined;
247  }
```

but most importantly we all are going to be using it for its agent or interactive mode:

Let's check out their **website** **(https://www.cursor.com/)** .

So fast that even Cursor itself can't remember!

Now this is a new kind of problem! These code editors are **so** fast that even the large LLMs they are using, sometimes forget what it was actually working on.

This presents itself as an amazing opportunity for us to learn GIT. Now we already had a session on **GIT** (https://www.youtube.com/watch?v=DL9uJKqugjc) and Python 101, but I would like to introduce this amazing **GAME** (https://learngitbranching.js.org/) that allows you to learn GIT in a refreshing way! Let me play a few stages, and hope that you'll finish it up yourself later!

The way you need to use Cursor is, moment you have made "any" change, you'll need to "commit" those changes. It doesn't cost much but the time savings are going to be amazing!

You also need to create a **Google AI Studio account** (https://aistudio.google.com/), and get the API keys for Gemini Flash. Also use AntiGravity (hunch says Cursor won't be able to make a comeback anytime **soon** (https://www.linkedin.com/posts/tschlottke_cursor-went-from-62-growth-to-18-in-6-activity-7404158026494152705-MhXB/)).

Consumers vs Producers

Before we start I want to share something that is at the core of TSAI courses.

They are HARD.

They need DISCIPLINE.

They need YOU to LOVE yourself.

There are a lot of courses and tutorials that could teach us how to become a consumer of LLMs, models, agents, apps, libraries, tools and a lot many more things. But TSAI's courses are designed to make you **producers**; produces of exactly the things like above.

In our context, if you want to know "how to make agents" or "agentic framework" or "GenAI tools", then this course if for you. Of while making these you'll also become great at using them as well. 😊

What are AI models? Let's start with a little bit of math.

AI models are sets of numbers that help turn an input into an output. During training, the model learns how to give right kind of output for each specific input.

Here are the some simple steps:

Problem Set up:

Equation: $y = mx + 4 = Wx + 4$

Model's Goal: Find m that fits (x, y): (1, 6) and (2, 8)
Loss: We'll use L1 loss: $Loss = |y - y_{pred}|$
Learning Rate: $\eta = 0.5$

1. Initialise m (step 0)

For x = 1, y = 6, but: $m = 0$

$y_{pred} = mx + 4 = 0 \cdot 1 + 4 = 4$ << this is a linear problem, we need LLM/ML to solve non-linear problems.

$Loss = |6 - 4| = 2$ << measure of **how good our models are (https://finbarr.ca/llms-not-trained-enough/)**.

$\frac{\partial Loss}{\partial m} = -x.\,sign\,(y - y_{pred}) = -1.sign\,(6 - 4) = -1$ << This is the error signal, also called **back-propagation (http://youtube.com/watch?v=Ilg3gGewQ5U&vl=en)** gradients.

$m' = m - \eta \frac{\partial Loss}{\partial m} = 0 - 0.5 \times (-1) = 0.5$ << our new value of m. We just had 1 step. Training an LLM would have millions of such updates.

2. Check with updated m (0.5)

For x = 1, y = 6, but: $m = 0.5$

$y_{pred} = mx + 4 = 0.5 \cdot 1 + 4 = 4.5$

$Loss = |6 - 4.5| = 1.5$

$\frac{\partial Loss}{\partial m} = -x.\,sign\,(y - y_{pred}) = -1.sign\,(6 - 4.5) = -1$

$m' = m - \eta \frac{\partial Loss}{\partial m} = 0.5 - 0.5 \times (-1) = 1.0$

3. Check with updated m (1)

For x = 1, y = 6, but:

$m = 1$

$y_{pred} = mx + 4 = 1 \cdot 1 + 4 = 5$

$$Loss = |6 - 5| = 1$$

$$\frac{\partial Loss}{\partial m} = -x.\,sign\,(y - y_{pred}) = -1.sign\,(6 - 5) = -1$$

$$m' = m - \eta\frac{\partial Loss}{\partial m} = 1 - 0.5 \times (-1) = 1.5$$

4. Check with updated m (1.5)

For x = 1, y = 6, but:

$$m = 1.5$$

$$y_{pred} = mx + 4 = 1.5 \cdot 1 + 4 = 5.5$$

$$Loss = |6 - 5.5| = 0.5$$

$$\frac{\partial Loss}{\partial m} = -x.\,sign\,(y - y_{pred}) = -1.sign\,(6 - 4) = -1$$

$$m' = m - \eta\frac{\partial Loss}{\partial m} = 1.5 - 0.5 \times (-1) = 2.0$$

5. Check with updated m (2.0)

For x = 1, y = 6, but:

$$m = 2$$

$$y_{pred} = mx + 4 = 2 \cdot 1 + 4 = 6$$

$$Loss = |6 - 6| = 0$$

$$\frac{\partial Loss}{\partial m} = -x.\,sign\,(y - y_{pred}) = -1.sign\,(6 - 6) = -1 \cdot 0 = 0$$ << We have a theoretical limitation on how low the loss can be for LLMs/ML

$$m' = m - \eta\frac{\partial Loss}{\partial m} = 2 - 0.5 \times (0) = 2$$

6. Publish your paper!

This really is what all is there to a neural network! Here we were solving very simple problem, but you now have the basic understanding of how neural networks work.

But

and a real but...., problems in life are not linear in nature. Our equation:

$$y = mx + 4$$

This equation was linear. Even a circle:

$$x^2 + y^2 = 4$$

is non-linear. More Examples..

$$f(x) = x^2$$
$$f(x) = \sigma(x)$$
$$f(x) = \max(0, x)$$

And we can't keep coming up with equations for each problem that we can identify. Then in 1989 we had a breakthrough!!

Universal Approximation Theorem
(https://en.wikipedia.org/wiki/Universal_approximation_theorem)

I am going to change a few words to its relevant to us. It was proven in 1989 that matrices with a **particular set of (activated) values** and matrix multiplications can represent **any** function! And these activation themselves must be Non-linear.

Basically, for complex problems..

y = mx + c wont work. We need

y = non-linear-function(x + c) + k.

They just couldn't figure out how to find these "**particular set of values**". 😅

Then came Mr. Geoffrey Hinton. But a bit of history:

Neural Networks were invented in 1943, but till 1989, people never believed that they could solve "all" problems and are just pattern matchers or statistical classifiers.

UAT's contribution was a break this notion.

Backprop was already invented by the 1960s, and in heavy use by 1974, but only by Geoffrey Hinton's application to DNNs in 1986 made backpropagation popular. **Yann LeCun (https://techcrunch.com/2026/03/09/yann-lecuns-ami-labs-raises-1-03-billion-to-build-world-models/)** applied it to Convolutions in the late 1980s.

So we had parallel fields finally colliding. UAT proved NNs can solve anything, we these people knew how to find "those particular values" and the field exploded.... at snail's pace.

Problems

Neural networks are values in matrices in the end, and we need to multiply them, a lot of them. GPU's were focused on Graphics, games, not solving these problems specifically. They could, but nobody had solved it they was Alex did!

He used NVIDIA GTX 580 GPU, which was extremely unconventional in 2012. He literally made NVIDIA this Trillion USD company today, and in return he received nothing.

He not only used the GPU, he used 2 of them! This was a big deal, and the start of GPU parallelisation.

After the AI Winter (from 1990s to 2010s), many researchers had given up on deep networks. Alex's huge win at the 2012 ImageNet competition (15% error drop!) forced the AI community to take deep learning seriously again!

He also worked on careful model optimisation, weight initialisation, dropout, and most importantly, released the code publicly.

After his PhD, he joined a startup funded by Hinton (Google Brain), Krizhevsky (Google Brain) and Sutskever (OpenAI). This startup was acquired by Google in 2013. He worked at Google for a bit, but left the AI research world around 2017, citing burnout and dissatisfaction with the field. No public presence anymore!

I am telling you this story because if you think life is hard, then Alex has lived it already.

I am also telling you this because all the above work happened in Canada, then the US, and Hinton was British, Alex was Ukrainian, Suskever was Israeli, and LeCun was French. Our country is missing from this list.

The field progressed smoothly, but by 2016 there were signs of stagnation.

This is how the world looked by 2016:

1. Text, image, audio, all had different neural networks to solve related problems
2. Every AI model needed data to be labeled and annotated
3. We couldn't make models bigger for images and audio
4. We couldn't make models work on longer texts

Problems that "Attention is all you need" solved

Of course we are going to talk about Transformer ultimately, but we need to understand what this paper solved along with introducing Transformers:

1. Labelled dataset is NOT required!!!!!!!
2. Same architecture for "any" problem! Finally, the generalised UAT
3. Model can be scaled AS MUCH money as **one can (https://openai.com/index/accelerating-the-next-phase-ai/)** raise 😅

4. Simplest possible architecture and training strategy!
5. Models are interpretable!! One can "see" why the model did what it did!

An intuition I would like to pass on to you, somehow 😅

If it is possible to make the model "decide" then let the model decide and don't act like **him (https://indianmemetemplates.com/wp-content/uploads/2019/01/Kabhi-kabhi-lagta-hai-apun-hi-bhagwan-hai.jpg)** .

And literally, whenever researchers hand over control to the model itself (rather than hand-coding human intuition), it almost always leads to breakthroughs!!!

Here are a few examples:

- Word Embeddings (2003)
- Neural Architecture Search (2014)
- Adam Optimizer (2014)
- Batch Normalization (2015)
- Attention (2015)
- Transformers (2017)
- VQ-VAE (2017)
- RLHF (2018-2020)
- Diffusion Models (2020)
- CLIP (2021)
- LoRA (2021)

Going back to transformers, think about what we were trying to do "inside" the neural networks. When looking at this image:

we wanted the model to focus on the dog, and say there is a dog in the image...

We wanted to model to "attend" or focus on dog, and from 2012-2017 we tried to solve this problem in numerous ways.

We were partially successful.

You need to understand what was wrong in this approach. We want model to do what "we" want, in the way "we" want.

You need to remember that this approach does not work!

In AIAYN paper, **Vasvani Ji (https://images.squarespace-cdn.com/content/v1/62ec2bc76a27db7b37a2b32f/2eb92ce2-a3c0-495b-8b1b-3303e0adbcf8/ashish-vaswani.jpg)**, allowed the model to figure this out, the way it wanted!

Now this is how the whole Transformer Model (with Encoder on the left and Decoder on the right) looks like:

but today we use way simpler models:

It looks like the block on the left above (called Encoder there), but we will call it Decoder (to confuse you). 😂

There's a lot going on in this architecture but first we need to clear the GRE hurdle:

- Input: a sequence of:
  - text, represented via words (we'll talk about tokens later)
  - image, represented via patches of images
  - audio, represented via patches of **audio (https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEgLBj7HiE4JZhOjRMZw2PdpCSaIQnLEYRv_bY21rHlikUUjQAvs982fuXKJwznwLtwgmMYTSgE2M-NrGdB7X_ALZlEsUOfe63HuIdppfbAziJm32foIMhGX-FEw2e-PnmmGH42OgPnyU2Y/s0/yamnet_animation+%25282%2529.gif)** converted into **mel-spectogram (https://ars.els-cdn.com/content/image/1-s2.0-S0045790624000284-gr3.jpg)**

  - video, sequence of patches of video frames
  - time-series data, a sequence of time-stamped numerical values
  - **DNA (https://engineering.berkeley.edu/news/2025/02/new-ai-breakthrough-can-model-and-design-genetic-code-across-all-domains-of-life/)** , sequence of base pairs (GCTA)
  - code, sequence of words, and special symbols

- Embeddings: each of the item above needs to be represented into something that a model can understand:
  - it cannot be just a number as that doesn't have a lot of information
  - how about something like this:

  - Problem here is, in how many ways can we decide the top row (also called embedding dimensions), and how do we find the right ones!
  - We solved it in 2003. We allow the computer to determine what these dimensions stand for. And unlike us (who think in discrete values, like 1, 2, 3, love, hate), AI models can have extremely complex and compact representations (for example, think of 15 vs 1111, and embeddings can be 0.1, -0.56, 0.00012, -0.5423)

- We now a days only tell our model "total dimensions", larger the better:

- **Position Encoding**: we "tag" our sequence inputs with their positional/location values:
  - this is required as when we send a lot of data, the model needs some way to identify what came before or later.

- **Attention**: the core algorithm that has made all of this development possible. This algorithm:
  - evolves the meaning of a word based on its surrounding work
    - *Dhoni is nothing less than a tiger. And this tiger won India a world cup!* The meaning of the word ***tiger*** is slightly different in both the sentences.

      Also analyse this:

      I went to the **bank**, which was next to the **bank** of Ganges, and on the left was a **banking** road.

      Attention mechanism identifies the correct meaning of the word based on the text around it
      - finds which part of the sequence is important, and what it should focus on. In the above sentence, if we ask "Who was the Tiger?", LLMs today can tell us, that we were referring to Dhoni.
      - has neural networks inside not shown

- **Multi-head**: we saw above that our "dimensions" can run into 10000s! We don't want to deal with all of them at the same time. So we dissect them, say into pieces of 128, or 258, and then process them. Helps in Parallel Processing and "contextuality".

- **Add**: addition, like 1 + 4 = 5 😎 see, not everything is new here

- **Norm**: normalizes things:
  - We like everything to be between -1 and +1, and we are adding things.. so 0.9, -0.8, 0.4 and 0.5, -0.5, 0.8 can become 1.4, -1.3 and 1.2. We convert that to 1, -1 and 0.85.

- **Feed Forward**: remember we "dissected" out embeddings? Here we combine them to mix "context" and build the whole thing (can't process contexts in isolation)
- **Nx**: Repeat N times

- **Linear**: Final Neural network Layer, that predicts

- **Softmax**: Prediction can be like 5, 0.4, 1, 2.3, 4.5.. Softmax converts that into 0.58, 0.0059, 0.0108, 0.0395, 0.3563, which sums up to 1

- **Output Probabilities**: Above numbers only.

Today was a "foundation" session. And I am hoping you're getting a good feel about things. We need a couple of sessions to understand Transformers or LLMs, so hang on.

Assignment

- Let's make something. Now I am going to show you how to make a Chrome Plugin without knowing a lot of stuff. See and learn.

- Your assignment is to make something unique and really useful. It should be something that you CAN use. This course is about learning how to make things, so let us remove any hesitation that we may have. Here are few ideas:
  - Show unread emails from "all" email accounts (I have like 10) in a list
  - Show Andrej Karpathy's latest like or tweet
  - Translate a few lines on the current page into German or Kannada
  - Show the latest TV series on "all" OTT networks
  - A Chrome Screen Saver that shows Jokes on AI or Beautiful Quotes
  - A plugin that finds the price of the product you're looking at on Amazon on different websites
  - Dark or Un-Dark Themer
  - Fill in the form
- Once done, upload a short YouTube video and share link with me!
- Scores are subjective, based on how much effort you've put in or how useful that plugin would be.

Video

Studio (**Transcript (https://canvas.instructure.com/courses/14531601/files/343641755?wrap=1)** **(https://canvas.instructure.com/courses/14531601/files/343641755/download?download_frd=1)**)

GMeet